VIETNAM NATIONAL UNIVERSITY - HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING

ADVANCED PROGRAMMING - CO2039

ASSIGNMENT 1

MINI TRADING PLATFORM

Author: MEng. Tran Truong Tuan Phat

HO CHI MINH CITY, 03/2026

Contents

1 Introduction **2**
1.1 Key Domain Concepts . . . 2
1.2 How the System Works . . . 2
1.3 Why These Design Patterns? . . . 3
1.4 Provided Starter Code . . . 3

2 Overview **3**

3 Part A — Instrument Hierarchy (Problems 1–3) **4**
3.1 Problem 1: Instrument and Stock (10 pts) . . . 4
3.2 Problem 2: Bond (5 pts) . . . 5
3.3 Problem 3: Derivative, Option, and Future (5 pts) . . . 5

4 Part B — Interfaces (Problems 4–5) **6**
4.1 Problem 4: Strategy Pattern–PricingStrategy (15 pts) . . . 6
4.2 Problem 5: Tradeable and Priceable (15 pts) . . . 7

5 Part C — Portfolio Management (Problem 6) **8**
5.1 Problem 6: Position and Portfolio with Observer Pattern (20 pts) . . . 8

6 Part D — Generic Risk Analysis (Problem 7) **9**
6.1 Problem 7: RiskAnalyzer (15 pts) . . . 9

7 Part E — Visitor Pattern (Problem 8) **10**
7.1 Problem 8: InstrumentVisitor and TaxReportVisitor (15 pts) . . . 10

8 Submission **11**

9 Grading Summary **11**

10 Other Regulations **12**

1 Introduction

Modern financial markets rely on sophisticated software systems to manage diverse financial instruments—from equities and fixed-income securities to derivative contracts such as options and futures. These platforms must handle real-time price updates, portfolio management, risk analysis, and regulatory reporting, all while maintaining clean separation of concerns and extensibility.

In this assignment, you will design and implement a **mini online trading platform** that synthesizes all OOP concepts covered throughout the course—class hierarchies, abstract classes, interfaces, generics, and multiple design patterns—into a single, cohesive application.

1.1 Key Domain Concepts

Term	Description
Instrument	A tradeable financial asset (e.g., stock, bond, option, future)
Stock	An equity instrument representing ownership in a corporation
Bond	A fixed-income instrument representing a loan, paying periodic coupons
Option	A derivative giving the right (but not obligation) to buy/sell an asset
Future	A derivative obligating the buyer to purchase an asset at a set date/price
Tax Liability	The total amount of tax owed on investments
Position	A holding of an instrument—quantity owned and average cost
Portfolio	A collection of positions owned by an investor
Market Value	Current worth: quantity × current price
Unrealized P&L	Profit/loss if the position were closed now
Risk Score	Numeric measure (0–10) of instrument risk
Fair Value	Estimated price computed by a pricing strategy

1.2 How the System Works

Imagine a typical trading session on your platform:

1. **Instruments are created.** A portfolio manager sets up various financial instruments—`Stock` objects like "AAPL" with a market cap, `Bond` objects like "US10Y" with a coupon rate, and derivatives such as call `Options` on a stock. Each instrument knows how to report its own **risk score** and **asset class** through polymorphism.

2. **A portfolio is built.** The manager creates a `Portfolio` and adds positions (e.g., 100 shares of AAPL at $150 cost basis). The portfolio *automatically notifies* any registered observers—such as a logging system or a dashboard—whenever a position is added or removed.

3. **Markets move.** Instrument prices change throughout the day. The manager calls `revalueAll()` with a `PricingStrategy`. The platform can swap between a `SimplePricingStrategy` (flat 5% premium) or a `RiskAdjustedPricingStrategy`

(premium scales with risk) *without changing any portfolio code*—this is the Strategy pattern at work.

4. **Portfolio analysis.** At any time, the manager can query `totalMarketValue()`, `totalUnrealizedPnL()`, sort positions by value, or see allocation percentages by asset class. The generic `RiskAnalyzer<T>` can analyze any subset of instruments.

5. **Tax reporting.** At year-end, a `TaxReportVisitor` traverses all instruments in the portfolio. Each instrument "accepts" the visitor and the visitor applies the correct tax rate—15% for stocks, 30% for bond coupons, 20% for derivatives. New report types can be added by creating new visitors *without modifying any instrument class*.

1.3 Why These Design Patterns?

Each design pattern in this assignment solves a specific architectural problem. The table below explains *why* each pattern was chosen:

Pattern	Problem	Solution
Strategy	Runtime-swappable pricing models	`PricingStrategy` interface; algorithms as separate classes. Portfolio calls `calculateFairValue()` generically.
Observer	External systems react to changes	`Portfolio` broadcasts events; observers register for automatic updates.
Visitor	Add analytics without modifying hierarchy	Each instrument has `accept()`; visitor implements type-specific logic via double dispatch.

1.4 Provided Starter Code

You are provided with skeleton Java files containing class declarations, field definitions, and method signatures. Each method body contains a `throw new UnsupportedOperationException("TODO: ...")` stub that you must replace with your implementation.

Important

Do **not** modify method signatures, class names, or the test file. Your implementation must compile and pass all provided tests without changes to the public API.

2 Overview

You will build a **mini online trading platform** step-by-step across 8 problems. Each problem builds on classes from earlier ones. The platform manages financial instruments, portfolios, risk analysis, and tax reporting.

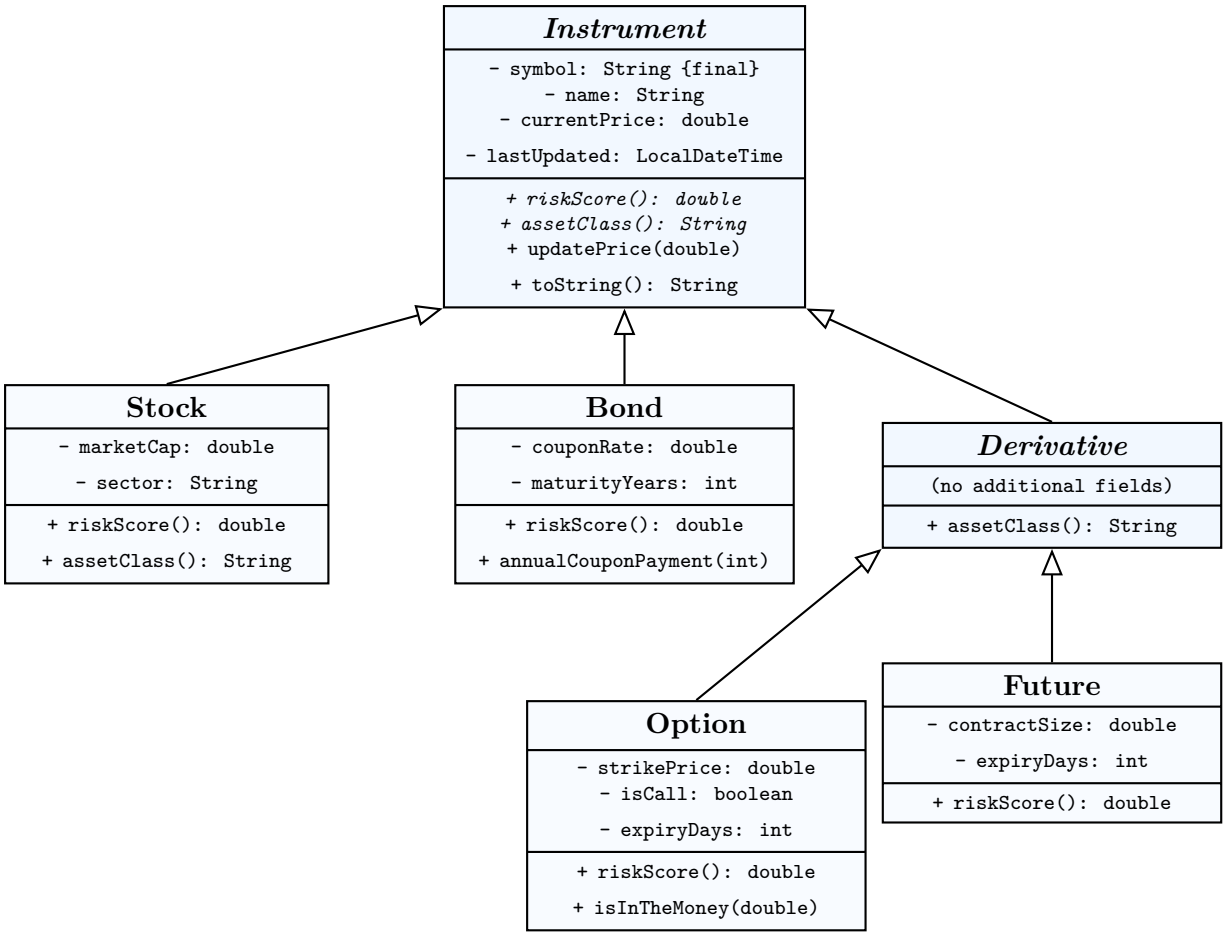
Assignment Structure

1. **Part A (Prob. 1–3):** Instrument class hierarchy — inheritance and polymorphism
2. **Part B (Prob. 4–5):** Interfaces — `PricingStrategy`, `Tradeable`,

`Priceable`

3. **Part C (Prob. 6):** Portfolio management — Observer pattern, positions, pricing
4. **Part D (Prob. 7):** Generic risk analysis — `RiskAnalyzer<T>`
5. **Part E (Prob. 8):** Visitor pattern — `TaxReportVisitor`

3 Part A — Instrument Hierarchy (Problems 1–3)



Italic names = abstract classes.

3.1 Problem 1: Instrument and Stock (10 pts)

Description: In this problem, you have to implement:

- `Instrument`: A generic tradeable financial asset. Key properties include `symbol` (unique identifier), `name`, `currentPrice`, and `lastUpdated` timestamp. It defines common behavior like updating prices and calculating risk scores.
- `Stock`: An equity instrument representing corporation ownership. It introduces `marketCap` (total market value of the company's shares) and `sector` (e.g., Technology, Healthcare) to evaluate its specific risk score.

- **Example:** Apple Inc. (`AAPL`) is a stock in the Technology sector with a current price of $150 and a massive market cap, meaning it carries a generally lower risk score compared to small start-ups.

Class	Requirements
`Instrument`	• Abstract class with fields: `symbol` (final), `name`, `currentPrice`, `lastUpdated`. • Abstract methods: `riskScore()`, `assetClass()`. • `updatePrice(double)`: throws `IllegalArgumentException` if negative; updates `lastUpdated`. • `toString()`: `"<ClassName>[symbol=<s>, price=<p>, risk=<r>]"`
`Stock`	• Fields: `marketCap` (double), `sector` (String). • `riskScore()`: marketCap $< 10^9 \rightarrow 7.5$; $< 10 \times 10^9 \rightarrow 5.0$; else 3.0. • `assetClass()`: `"EQUITY"`

Files to upload: `Instrument.java`, `Stock.java`

3.2 Problem 2: Bond (5 pts)

Description: In this problem, you have to implement `Bond` – A loan to an entity that pays periodic interest (coupons). Key fields include `couponRate` (annual interest rate percentage) and `maturityYears` (time until the principal is repaid), which are used to determine its risk and calculate annual payouts through `annualCouponPayment(int units)`.

- **Example:** A 10-year US Treasury bond (`US10Y`) with a face value (current price) of $1,000 and a 5% coupon rate will pay the bondholder $50 annually for 10 years until maturity.

Class	Requirements
`Bond`	• Fields: `couponRate` (double), `maturityYears` (int). • `riskScore()`: maturityYears $> 10 \rightarrow 4.0$; else 2.0. • `assetClass()`: `"FIXED_INCOME"` • `annualCouponPayment(int units)`: `units*currentPrice*couponRate/100`

Files to upload: `Instrument.java`, `Bond.java`

3.3 Problem 3: Derivative, Option, and Future (5 pts)

Description: In this problem, you have to implement `Derivative`, `Option`, `Future`. These are financial contracts whose value relies on an underlying asset.

- `Option`: Gives the right, but not the obligation, to buy (`isCall=true`) or sell (`isCall=false`) at a specific `strikePrice` before the `expiryDays`. The `isInTheMoney(double spotPrice)` method checks if exercising the option is currently profitable compared to the market `spotPrice`.
  - **Example:** A call option on Apple stock with a strike price of $160. If the market spot price of Apple rises to $170 before the option expires (`isInTheMoney = true`), you can exercise the option to buy the stock for only $160, securing a profit.

- `Future`: An obligation to buy or sell a specific `contractSize` of an asset after `expiryDays`.
  - **Example:** A gold future contract obligating you to buy 100 ounces of gold in 30 days at today's agreed price, regardless of whether gold goes up or down.

Class	Requirements
`Derivative`	• Abstract. `assetClass()`: `"DERIVATIVE"`
`Option`	• Fields: `strikePrice`, `isCall`, `expiryDays`. • `riskScore()`: 8.5. • `isInTheMoney(double spotPrice)`: isCall? → `spotPrice>strikePrice`; else `spotPrice<strikePrice`.
`Future`	• Fields: `contractSize`, `expiryDays`. • `riskScore()`: 8.5.

Files to upload: `Instrument.java`, `Derivative.java`, `Option.java`, `Future.java`

4 Part B — Interfaces (Problems 4–5)

4.1 Problem 4: Strategy Pattern–PricingStrategy (15 pts)

Description: In this problem, you have to implement the Strategy pattern for pricing different instruments.

- `PricingStrategy`: An interface defining `calculateFairValue(Instrument)` to compute an instrument's estimated price, and `strategyName()` to identify the algorithm.
- `SimplePricingStrategy`: A concrete strategy that applies a flat 5% premium.
  - **Example:** If a stock's current price is \$100, this strategy calculates a fair value of \$105.
- `RiskAdjustedPricingStrategy`: A dynamic strategy that scales the premium based on the instrument's risk score (1% per risk point).
  - **Example:** If a bond has a price of \$1,000 and a risk score of 4.0, the fair value becomes $\$1,000 \times (1 + 0.04) = \$1,040$.

Interface / Class	Requirements
`PricingStrategy`	Interface with: • `double calculateFairValue(Instrument)` • `String strategyName()`
`SimplePricingStrategy`	• `calculateFairValue`: `currentPrice * 1.05` (5% premium) • `strategyName()`: `"SimplePricingStrategy"`
`RiskAdjustedPricingStrategy`	• `calculateFairValue`: `currentPrice * (1 + 0.01 * riskScore())` • `strategyName()`: `"RiskAdjustedPricingStrategy"`

Files to upload: `Instrument.java`, `Stock.java`, `PricingStrategy.java`, `SimplePricingStrategy.java`, `RiskAdjustedPricingStrategy.java`

4.2 Problem 5: Tradeable and Priceable (15 pts)

Description: In this problem, you have to implement two interfaces that define the capabilities of financial instruments:

- `Tradeable`: Exposes methods representing an asset's market presence: `getSymbol()` returns its ticker, and `isAvailableForTrading()` tells if it can be bought/sold.
  - **Example:** Calling `getTradingInfo()` on AAPL might output `"Tradeable: AAPL at $150.0 (Available)"`.
- `Priceable`: Focuses on price fluctuations over time. It provides absolute and percentage changes compared to previous historical prices.
  - **Example:** If an asset's previous price was $100 and today's is $105, `getPriceChange()` returns $5.0, while `getPriceChangePercent()` returns 5.0%.

Define two interfaces and modify `Instrument` to implement both.

Interface	Methods
`Tradeable`	• `String getSymbol()` • `double getCurrentPriceValue()` • `boolean isAvailableForTrading()` • `default String getTradingInfo()` — returns formatted trading summary
`Priceable`	• `double getCurrentPriceValue()` • `double getPriceChange(double previousPrice)` — absolute change • `double getPriceChangePercent(double previousPrice)` — percentage change

Instrument must implement Tradeable and Priceable

Modify your `Instrument` class declaration to: `public abstract class Instrument implements Tradeable, Priceable`.

Provide **concrete implementations** of these methods in `Instrument`:

- `getPriceChange(double previousPrice)` — returns `currentPrice - previousPrice`
- `getPriceChangePercent(double previousPrice)` — returns `((currentPrice - previousPrice) / previousPrice) * 100`
- `isAvailableForTrading()` — returns `true`

Note: `getSymbol()` and `getCurrentPriceValue()` are already defined in `Instrument` from Problem 1. These concepts allow instruments to be dynamically priced and traded in the market.

Files to upload: `Tradeable.java`, `Priceable.java`, `Instrument.java`, `Stock.java`

5 Part C — Portfolio Management (Problem 6)

5.1 Problem 6: Position and Portfolio with Observer Pattern (20 pts)

Description: In this problem, you have to implement the Observer pattern to monitor a portfolio of positions.

- `Position`: Represents a holding of a specific financial instrument, tracking how many shares (`quantity`) you own and the average price you paid for them (`averageCostBasis`).
  - **Example:** You buy 10 shares of AAPL at $150. Your `Position` is `quantity = 10`, `averageCostBasis = 150`. If AAPL is now $160, `marketValue()` is $1600, and `unrealizedPnL()` is +$100.
- `Portfolio`: An investor's overarching account. It implements the Observable interface so other systems (like UI dashboards) can be notified when positions are added, removed, or revalued.
  - **Example:** When you call `addPosition()` to buy 5 shares of MSFT, the Portfolio creates/updates the MSFT Position and notifies observers with `"ADDED: MSFT x5"`, so a connected UI can update its display immediately.

Note: You can modify `Instrument` and/or its inherited classes for this problem to implement Observer pattern correctly.

First, define two new interfaces for the Observer pattern:

Interface	Methods
`Observable<T>`	• `void addObserver(Observer<T>)` • `void removeObserver(Observer<T>)` • `void notifyObservers(T event)`
`Observer<T>`	• `void onEvent(T event)`

Then implement `Position`, `Portfolio`, and `PositionNotFoundException`:

Class	Requirements
`Position`	• Fields: `instrument` (Instrument), `quantity` (int), `averageCostBasis` (double). • `marketValue()`: `quantity * currentPrice` • `unrealizedPnL()`: `marketValue() - quantity * averageCostBasis` • `addQuantity(int qty, double costBasis)`: merges with weighted average cost
`PositionNotFound-Exception`	• Checked exception (`extends Exception`). • Constructor takes a `String` message.
`Portfolio`	• Implements `Observable<String>`. Fields: `portfolioId`, `ownerName`, positions list, observers list. • `addPosition(Instrument, int, double)`: creates or merges; notifies `"ADDED: <symbol> x<qty>"` • `removePosition(String)`: throws `PositionNotFoundException`; notifies `"REMOVED: <symbol>"` • `getPosition(String)`: throws `PositionNotFoundException` if not found • `totalMarketValue()`, `totalUnrealizedPnL()`: sum across all positions • `getPositionsSortedByValue()`: descending by market value • `revalueAll(PricingStrategy)`: updates instrument prices via strategy; notifies `"REVALUED: <strategyName>"`

Files to upload: Upload necessary files.

6 Part D — Generic Risk Analysis (Problem 7)

6.1 Problem 7: RiskAnalyzer (15 pts)

Description: In this problem, you will implement a Generic class to perform statistical analysis on risk profiles.

- `RiskAnalyzer<T>`: Analyzes a collection of instruments and returns metrics like the average risk or the instrument with the highest risk score.
  - **Example:** If a `RiskAnalyzer<Stock>` contains AAPL (risk 3.0) and a tiny startup stock (risk 7.5), `averageRisk()` is 5.25, and `highestRisk()` returns the startup stock. The `getAboveRiskThreshold(5.0)` method would return a list containing only the startup stock.

Class	Requirements
`RiskAnalyzer<T extends Instrument>`	• Generic class with bounded type parameter. • `void add(T instrument)`: adds an instrument to the analyzer • `double averageRisk()`: average `riskScore()` of all instruments • `T highestRisk()`: instrument with the highest risk score • `T lowestRisk()`: instrument with the lowest risk score • `List<T> getAboveRiskThreshold(double threshold)`: all instruments with risk > threshold

Files to upload: Upload necessary files.

7 Part E — Visitor Pattern (Problem 8)

7.1 Problem 8: InstrumentVisitor and TaxReportVisitor (15 pts)

Description: In this problem, you have to implement the Visitor pattern, allowing new operations to be defined on the `Instrument` hierarchy without modifying the classes themselves.

- `InstrumentVisitor`: The interface defining a `visit()` method for every concrete instrument type.
- `TaxReportVisitor`: A concrete visitor that calculates tax liability differently depending on if the asset is a stock, bond, or derivative.
  - **Example:** When evaluating a Portfolio, you pass the `TaxReportVisitor` to each instrument's `accept()` method. A $100 Stock will calculate a $15 tax liability (15% of currentPrice), while an Option will calculate a $20 liability (20% of currentPrice).

Note: You can modify `Instrument` and/or its inherited classes for this problem to implement Visitor pattern correctly.

Interface / Class	Requirements
`InstrumentVisitor`	Interface with overloaded methods: • `void visit(Stock)` • `void visit(Bond)` • `void visit(Option)` • `void visit(Future)`

`TaxReportVisitor`	• Implements `InstrumentVisitor`. Computes tax liability:
	  ○ Stock: 15% of `currentPrice`
	  ○ Bond: 30% of annual coupon on 1 unit
	  ○ Option / Future: 20% of `currentPrice`
	• Accumulates `totalTaxLiability`.
	• `getTotalTaxLiability()`: returns accumulated total
	• `getReport()`: returns formatted breakdown string

Files to upload: Upload necessary files.

8 Submission

For each problem on Moodle CodeRunner, students only need to submit the **Java files** corresponding to that specific problem. Students are allowed to have a **maximum of up to 5 submissions**; however, only the final submission will be used for grading and evaluation.

Due to the possibility of system overload when many students submit simultaneously, it is strongly recommended to submit as early as possible. Late submissions are the student's responsibility. Once the submission deadline has passed, the system will be closed and no further submissions will be accepted. Submissions through other means will not be considered under any circumstances.

9 Grading Summary

Problem	Topic	Points
1	Instrument + Stock	10
2	Bond	5
3	Derivative, Option, Future	5
4	PricingStrategy (Strategy pattern)	15
5	Tradeable & Priceable interfaces	15
6	Position, Portfolio, Observer pattern	20
7	RiskAnalyzer`<T>` (Generics)	15
8	Visitor pattern + TaxReportVisitor	15
	Total	**100**

Note: A penalty policy applies to all problems, so please check your answers carefully before submitting them to the LMS. If you receive a runtime error, it means that your code either fails to compile or cannot finish execution within 2–3 seconds and you must revise your code.

10 Other Regulations

- Students must complete this assignment independently and must not allow others to copy their work. Any violation will be treated as academic dishonesty in accordance with university regulations.
- All decisions made by the instructors responsible for this assignment are final.
- Test cases will not be disclosed after grading.

——————————**END**——————————